Semantic dictionary for movie search

Helping language models understand what movie reviews mean

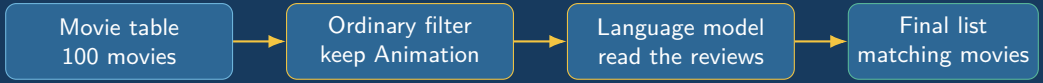
The task → the problem → the fix → the results

Master 2 project

July 2026

The task in one simple example

User question: “Which animated movies have reviews saying the film is funny?”



Structured filter

Checks facts already stored in columns: genre, year, runtime, director or title.

Semantic filter

Reads free text and decides whether a review really supports ideas such as funny, scary, original or well acted.

Goal: return as many correct movies as possible, without returning too many wrong ones, and without making unnecessary model calls.

What are the four solutions?

Solution	In plain language	Main difference
SUQL baseline	Turns the question into a database query, then asks a model about reviews	Uses the strong model for semantic decisions
SUQL V1	Same query idea, with a cheap model first	Sends uncertain cases to the strong model
Trummer baseline	Compares blocks of movies and reviews in semantic-join prompts	Many movie–review pairs can share one call
Trummer V1	Filters rows first, then uses a two-model batched join	Cheap routing + strong fallback + fixed batching

The comparison

All four solutions receive the same questions, movie data, reviews, model pair and semantic dictionary. This makes their quality, time and call counts directly comparable.

Experimental setting

	5q development suite	10q held-out suite
Questions	5 established tasks	10 newly constructed questions, unseen in 1q, 3q, or 5q
Data	IMDb movie facts and reviews; 12 correct answers/question	10 separate 100-movie datasets; 40 rows survive the ordinary filter; 12 correct answers/question
Repetitions	1	10
Models	Gemma 4 e2b cheap stage + Gemma 4 26b expensive stage	
Solutions	SUQL baseline, SUQL V1 cascade, Trummer baseline, Trummer V1 cascade	
Compute	Kraken H200; identical model pairing across all four implementations	

Why two test sets?

5q checks the implementation on known development questions. The 10 questions were newly written and had not appeared in the 1q, 3q or 5q sets. The 10q run contains 400 executions: 10 questions $\times$10 repetitions $\times$4 solutions.

Source: `benchmarks/5q/manifest.json`; `benchmarks/10q/manifest.json`

How the semantic dictionary was built

Keeping the test fair

- Movies are split into mining and holdout groups.
- The mining code never reads holdout review text.
- Mined words are hints, not automatic rules.

What is stored

- The meaning being searched for.
- Helpful words and phrases.
- Examples that support the meaning.
- Hard negatives: related text that does not support it.

Source: `semantic_dict/mine_semantic_dict.py`; `semantic_dict/categories.json`; `semantic_dict/README.md`

A small real part of the semantic dictionary

Entry: `praise_chemistry`

Stored JSON content, shortened only after the first examples

```
"template_type": "praise",
"attribute": "romantic chemistry",
"lexical_anchors": [
  {"term": "chemistry", "z_score": 9.80},
  {"term": "gates", "z_score": 7.44}, ...],
"few_shot_positive": [
  "...great chemistry between the love interests..."],
"few_shot_hard_negative": [
  "...a dose of drama...and romance, but never really ventures fully into it..."]
```

How to read it

"Chemistry" is a useful clue. "Romance" alone is not enough. The hard negative teaches the model the difference between a related topic and real praise of a relationship.

Source: semantic_dict/semantic_dict.json: praise_chemistry

How the dictionary was used in all four solutions

Rendered prompt context

```
MINED SEMANTIC GUIDELINE
(advisory context, not a keyword rule)
Category + predicate type + attribute
Mined anchors
Positive proxy exemplars
Hard-negative exemplars
```

Shared safety instructions

- Anchors suggest where to inspect; they never prove YES.
- Missing an anchor never proves NO.
- Judge only the current review.
- Copy evidence from the current review, not exemplars.

Implementation	Injection point	Evaluation unit
SUQL baseline / V1	semantic review judge	single review or routed review batch
Trummer baseline / V1	semantic join prompt	one prompt per block pair; multiple tuples per call

Coverage: all 5 development meanings; 5/10 held-out meanings (humor, acting, chemistry, excitement, ending). The other five questions use the normal prompt without a matching dictionary entry.

Source: `*/semantic_dict_context.py; Trummer/*/operators.py; Trummer/v1/cascade.py`

Prompt evolution 1 — the original short prompt

Original semantic decision prompt

```
Classify whether the review answers the question with Yes.
Return exactly one token:  1 or 0.
```

Why this was fragile

- It did not explain what counts as evidence.
- It did not say how to handle synonyms, indirect wording or uncertainty.
- Different model families interpreted the boundary differently.
- A confident-looking NO could remove a correct movie immediately.

Result: some model configurations rejected every candidate and returned zero final rows.

Prompt evolution 2 — the recall-first prompt

Prompt used after the first fix

```
You are a recall-first semantic filter for movie reviews.

Answer YES when there is any concrete review-specific evidence:
direct wording, a synonym/paraphrase, a described example,
or a reasonable implication.
Give borderline evidence the benefit of the doubt.
Answer NO when support is absent, generic, based only on genre/topic,
or the review never discusses the condition itself.
Do not infer evidence merely because the condition is not contradicted.

Return exactly YES or NO.
```

Change: the model now receives a clear decision boundary. Uncertain and negative cheap-model decisions can be checked by the stronger model.

Source: `fix/project SUQL/baseline/suql_engine.py: ANSWER_SYSTEM`

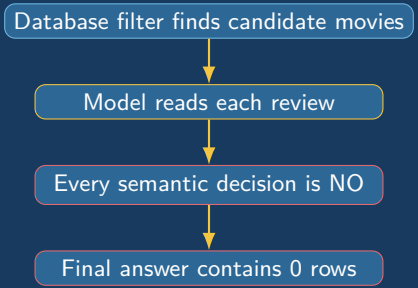
Final prompt with the semantic dictionary

Verbatim prompt structure used by Trummer V1

```
Classify every explicit candidate pair below.
Predicate: Which Romance movies ... praise the characters' chemistry?
MINED SEMANTIC GUIDELINE (advisory prompt context, not a keyword rule):
Category: praise_chemistry
Predicate type: praise
Attribute: romantic chemistry
Use the examples to distinguish actual predicate support from topic-only mentions.
Anchors suggest where to inspect, but an anchor alone never proves YES
and its absence never proves NO.
Mined lexical anchors: chemistry, gates, lincoln, chan, riley, ...
Positive proxy exemplars:
  + ...great chemistry between the love interests...
Hard-negative exemplars:
  - ...a dose of drama...and romance, but never really ventures fully into it...
Judge only the current review and copy evidence from it.
For every candidate extract verbatim evidence, give P(YES), and decide YES or NO.
CANDIDATE_1: Movie: [movie row] Review: [review text]
CANDIDATE_2: Movie: [movie row] Review: [review text]
Decisions:
```

Source: fix/project Trummer/v1/trummer_join/cascade.py; semantic_dict_context.py

Why did some runs return zero final rows?



The real problem

The database was not empty. The semantic stage was too strict, or its score could not be interpreted reliably. The pipeline treated that uncertainty like a certain rejection.

How the zero-row problem was tackled

1. **Clearer prompt:** define direct evidence, paraphrases, implications and topic-only negatives.
2. **Recall-first routing:** borderline cases are not deleted immediately.
3. **Stronger fallback:** uncertain cheap-model decisions go to Gemma 4 26B.
4. **Batch-safe output:** every candidate ID must appear exactly once.
5. **Monitoring:** count final rows and inspect missing decisions.

Evidence that the fix held

The final 10q experiment completed 400/400 runs with zero zero-row outputs and no tracebacks.

Why the old Gemma pipeline could return zero rows

What the old pipeline expected

- A one-token 1/0 answer.
- Comparable probabilities for both possible tokens.
- A signed confidence score: strong YES, uncertain, or strong NO.
- Only uncertain rows were sent to the larger model.

What could happen with Gemma/Ollama

- The API could expose only the probability of the token actually generated.
- The alternative token probability could be absent or not comparable.
- A generated 0 could look like a confident negative.
- That row then bypassed the 31B fallback and was deleted.

Why the larger model did not rescue the answer

The 31B model saw only rows placed in the uncertainty band. Rows mistakenly treated as "confident NO" never reached it. If this happened to every candidate, the final table was empty.

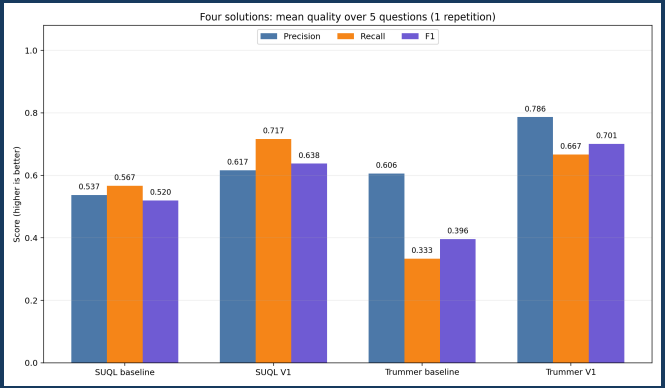
How the final design removed the mismatch

1. The cheap model can express **UNCERTAIN**; it is not forced to turn weak evidence into NO.
2. The prompt defines direct evidence, synonyms, implications and topic-only negatives.
3. A score is derived safely from the generated token; missing confidence is routed to the strong model.
4. Routing thresholds are calibrated; uncertain answers and missing scores go to the strong model.
5. The cascade has only two levels, so there are fewer places to lose a correct movie.
6. Batched prompts require every candidate ID exactly once, making missing decisions visible.

Outcome with Gemma 4 e2b $\rightarrow$ 26B

The held-out experiment completed 400 runs with no zero-row outputs. This shows compatibility for this tested model pair and prompt design.

5q results — quality



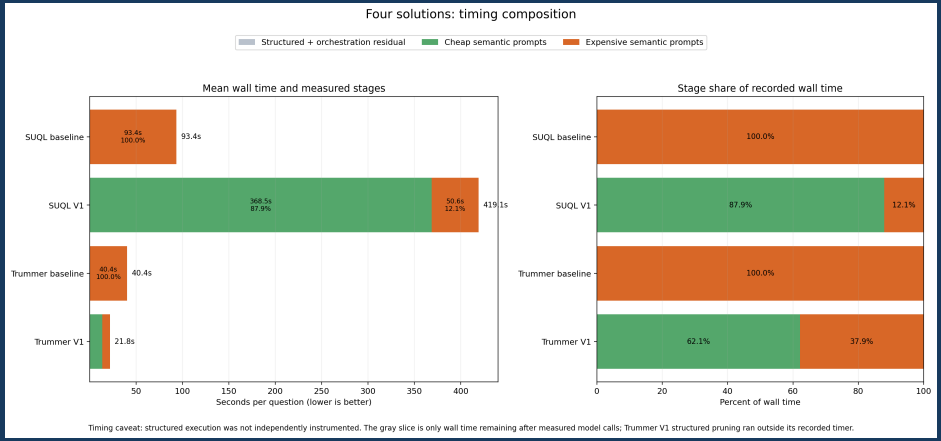
Method	P	R	F1
SUQL B	.537	.567	.520
SUQL V1	.617	.717	.638
Trummer B	.606	.333	.396
Trummer V1	.786	.667	.701

5q verdict

Trummer V1 gives the best F1 and precision. SUQL V1 gives the best recall.

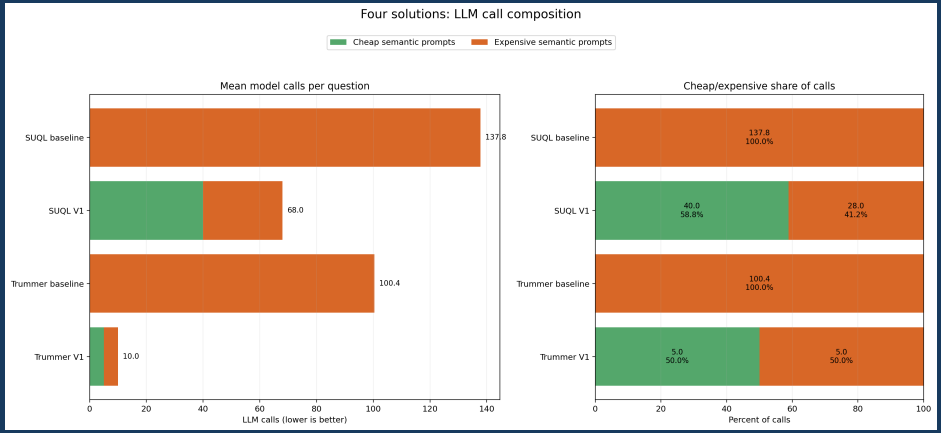
Source: `four_solutions_aggregate.csv; 5 questions, 1 repetition`

5q results — time spent at each stage



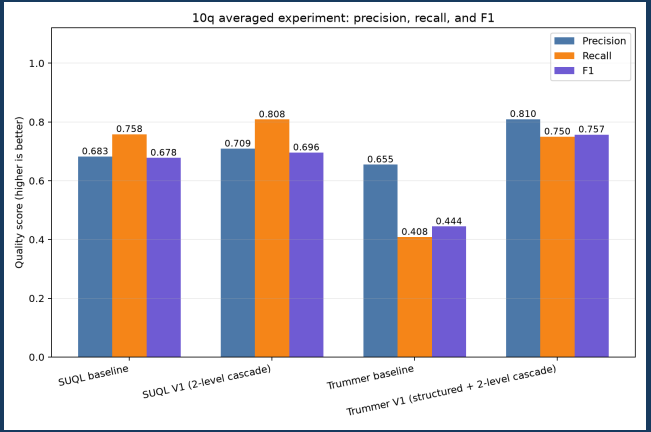
Trummer V1 is fastest: 21.8 seconds/run. SUQL V1 spends most of its 419.1 seconds in cheap semantic prompts.

5q results — number of model calls



Trummer V1 evaluates the candidate blocks in 10 calls/run: 5 cheap calls and 5 strong-model calls.

Held-out 10q results — quality



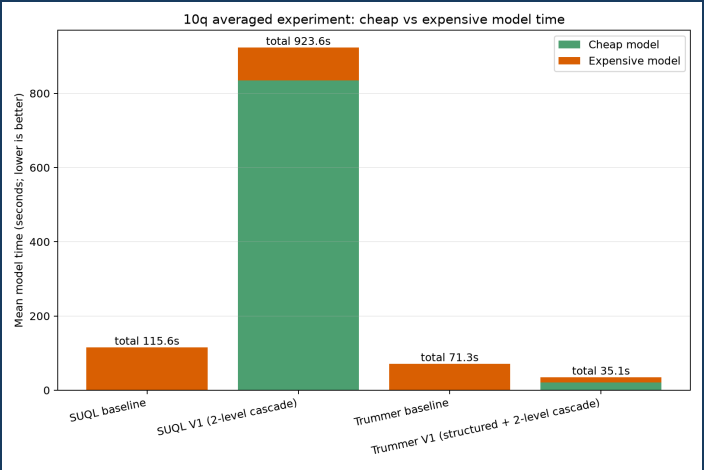
Method	P	R	F1
SUQL B	.683	.758	.678
SUQL V1	.709	.808	.696
Trummer B	.655	.408	.444
Trummer V1	.810	.750	.757

10q verdict

The 5q ranking generalizes:
Trummer V1 has the best F1;
SUQL V1 remains the recall leader.

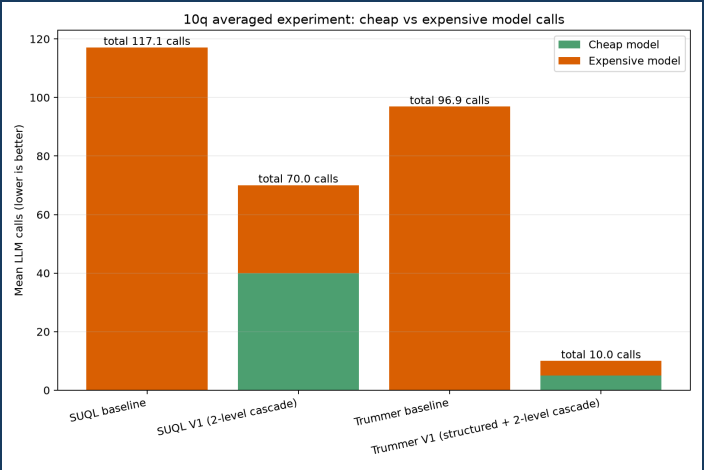
Source: 10q aggregate.csv; 10 newly constructed questions, 10 repetitions

Held-out 10q — time spent at each stage



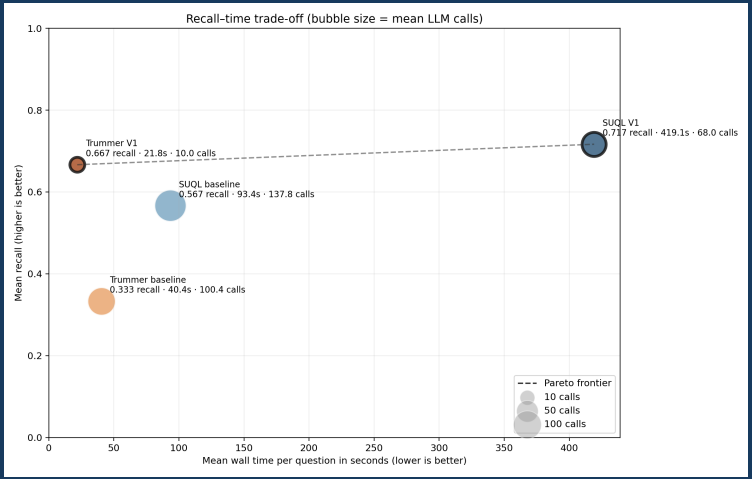
Trummer V1 remains fastest at 35.1 seconds/run. SUQL V1 averages 923.6 seconds because its cheap semantic stage is very slow.

Held-out 10q — number of model calls



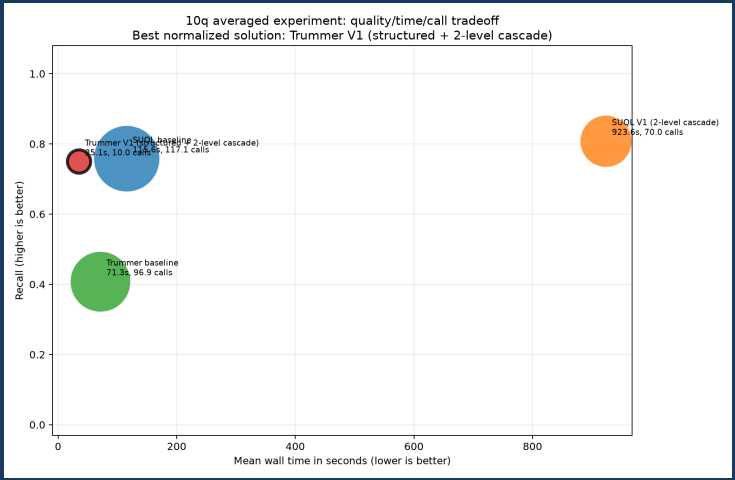
Trummer V1 still needs exactly 10 calls/run; the other implementations need about 70–117 calls.

5q implementation comparison — recall, time and calls



Better positions combine high recall with low time. Marker size represents the number of model calls.

10q implementation comparison — which is best?



Trummer V1 has the best overall balance: high recall, the shortest time, and the smallest call count. SUQL V1 has the highest recall but is much slower.

Final comparison: development to held-out

	5q, 1 rep			new 10q, 10 reps		
Method	Recall	F1	Time	Recall	F1	Time
SUQL baseline	.567	.520	93.4s	.758	.678	115.6s
SUQL V1	.717	.638	419.1s	.808	.696	923.6s
Trummer baseline	.333	.396	40.4s	.408	.444	71.3s
Trummer V1	.667	.701	21.8s	.750	.757	35.1s

Quality

V1 improves both families. SUQL V1 maximizes recall; Trummer V1 produces the strongest precision–recall balance.

Efficiency

Batching and structured pruning make Trummer V1 the fastest solution with exactly 10 model calls on both suites.

No zero-final-row outputs occurred in the 400-run held-out experiment.

Conclusions

1. The semantic dictionary gives the model useful examples and clues, while still asking it to understand the current review.
2. The new 10q test confirms the main 5q result on questions and movies not used before.
3. Trummer V1 is the best overall solution: best F1 (.757), best precision (.810), fastest time (35.1 s), and only 10 calls/run.
4. SUQL V1 finds the most correct movies: recall .808, but it is much slower.
5. Two choices mattered together: explain the decision clearly, and put several movie–review pairs in each model call.